Cognivia GmbH

Pflichtenheft

„Entwicklung einer Study-Planner-Anwendung in Java“

Stand:	01.06.2026
Auftraggeber:	InnoWeb Solutions GmbH Friedrich-List-Platz 1 01069 Dresden
Auftragnehmer:	Cognivia GmbH Moritzstraße 15 01097 Dresden
Ansprechpartner:	Projektteam „Study-Planner“ kontakt@cognivia.de www.cognivia.de
Autoren:	Yasamin Habibollahi Yara Dalloul Dana Lenzig Luise Schäffel Ina Savina

Inhaltsverzeichnis

1 Zielbestimmung 1
1.1 Muss-Kriterien 2
1.2 Kann-Kriterien 3
1.3 Abgrenzungskriterien 3
2 Produkteinsatz 4
2.1 Anwendungsbereich 4
2.2 Zielgruppen 4
2.3 Produktumgebung 4
2.3.1 Architektur 4
2.3.2 Technologie 5
2.3.3 Komponenten 5
2.3.4 Schnittstellen 5
2.4 Betriebsbedingungen 5
3 Produktfunktionen/Anforderungen 6
3.1 Funktionale Anforderungen 6
3.1.1 Beschreibung der FA mit Rollen innerhalb der Geschäftsprozesse 6
3.1.2 Aktivitäten mit Benutzerschnittstelle (UI) 7
3.1.3 Fachliches Klassendiagramm (domain Model) / Produktdaten 14
3.2 Nichtfunktionale Anforderungen 15
3.2.1 Benutzbarkeit 15
3.2.2 Zuverlässigkeit 15
3.2.3 Effizienz 15
3.2.4 Softwarewartung 15
3.2.5 Sicherheit 16
3.2.6 Normen 16
4 Testung 17
5 Monitoring/ Support bei Übergabe oder ähnlicher Leistungen 17
6 Dokumentation 18
6.1 Anwenderdokumentation 18
6.2 Administratorendokumentation 18
6.3 Entwicklerdokumentation 18
6.4 Weitere referenzierte Dokumente 19
7 Vorgehen 20
8 Entwicklungsumgebung 21
9 Glossar 22

1 Zielbestimmung

Die Study-Planner-Anwendung soll Studierende dabei unterstützen, ihren Studienalltag besser zu organisieren. Aktuell werden Termine und Lernzeiten oft in verschiedenen Tools festgehalten, wodurch leicht der Überblick verloren geht. Ziel ist eine lokal installierte Java-Anwendung mit einer einfachen, übersichtlichen Oberfläche, in der studienbezogene Termine und Lernblöcke verwaltet und mit Erinnerungen verknüpft werden können. Die Umsetzung orientiert sich an den Anforderungen des zugehörigen Lastenhefts.

1.1 Muss-Kriterien

MK-F-01	Modulverwaltung	Implementierung von **F1**: Die Anwendung muss Module mit Bezeichnung, Semester und optionaler Beschreibung anlegen, bearbeiten und löschen können.
MK-F-02	Terminverwaltung	Implementierung von **F2**: Zu Modulen und studienbezogenen Aktivitäten müssen Termine z. B. Vorlesungen, Prüfungen und Abgaben mit Datum und Uhrzeit erfasst, geändert, gelöscht sowie als „erledigt“ oder wieder als „offen“ markiert werden können. Der Status muss gespeichert und in der Benutzeroberfläche sichtbar dargestellt werden.
MK-F-03	Lernplanung	Implementierung von **F3**: Für einzelne Module müssen Lernblöcke mit Datum, Dauer und Modulzuordnung verwaltet werden können. Lernblöcke müssen zusätzlich als „erledigt“ oder wieder als „offen“ markiert werden können. Der Status muss gespeichert und in der Benutzeroberfläche sichtbar dargestellt werden.
MK-F-04	Kalenderansichten & Filter	Umsetzung von **F4** und **F5**: Das System muss Termine und Lernblöcke in mindestens einer Wochen- und einer Tagesansicht anzeigen und eine Filterung nach Zeitraum und Modul erlauben.
MK-F-05	Erinnerungen	Implementierung von **F6**: Die Anwendung muss Erinnerungen für ausgewählte Termine als Pop-up anzeigen. Der Vorlauf, z. B. ein Tag vorher, muss einstellbar sein.
MK-D-01	Datenspeicherung	Umsetzung von **F7**, **NF4** und **NF6**: Alle Planungsdaten (Module, Termine, Lernblöcke etc.) müssen lokal gespeichert und beim nächsten Start vollständig wieder geladen werden; Datenverlust bei normaler Nutzung ist zu vermeiden.
MK-T-01	Technologie/Plattform	Umsetzung von **NF3** und **NF5**: Die Anwendung muss in Java implementiert und auf aktuellen Versionen von Windows und Linux als Einzelplatzanwendung lauffähig sein.
MK-Q-01	Bedienbarkeit & Übersichtlichkeit	Umsetzung von **NF1** und **NF2**: Die Oberfläche muss übersichtlich sein; die Grundfunktionen sollen nach kurzer Einarbeitung (max. ~30 Minuten) bedienbar sein, und Deadlines sollen klar erkennbar hervorgehoben werden.

1.2 Kann-Kriterien

KK-IMP-01	Basis-Import/Export	Die Anwendung kann Planungsdaten, z. B. Module, Termine, Lernblöcke, in einer lokalen Datei exportieren und aus einer solchen Datei wieder importieren. Diese Funktion orientiert sich an Anforderung F8 und wird umgesetzt, sofern der Projektumfang dies zulässt.

1.3 Abgrenzungskriterien

AK-SYS-01	Keine mobile App	Die erste Version der Study-Planner-Anwendung wird nur als Desktop-Anwendung für Windows und Linux umgesetzt. Eine mobile App für Smartphone oder Tablet gehört nicht zum Lieferumfang.
AK-SYS-02	Keine Onlinesynchronisation	Es findet keine Synchronisation mit Online-Kalendern oder Cloud-Diensten statt. Export und Import erfolgen ausschließlich über lokale Dateien; eine Anbindung an externe Services ist in der aktuellen Version nicht vorgesehen.
AK-SYS-03	Kein Mehrbenutzerbetrieb	Die Anwendung wird als Einzelplatzlösung entwickelt. Eine gemeinsame Nutzung eines Datenbestands über Server oder Netzwerk ist nicht Teil dieses Projekts.

2 Produkteinsatz

2.1 Anwendungsbereich

Die Study-Planner-Anwendung wird im Umfeld von Hochschulen eingesetzt. Hauptnutzer sind Studierende, die ihren Studienalltag planen und strukturieren möchten. Typische Anwendungsfälle sind das Erfassen von Veranstaltungen (Vorlesungen, Übungen, Praktika), Prüfungen und Abgabefristen sowie das Planen von Lernblöcken. Die Software wird auf den persönlichen Rechnern der Studierenden installiert und unterstützt ausschließlich die individuelle Studien- und Lernplanung; Verwaltungsprozesse der Hochschule gehören nicht zum Einsatzbereich.

2.2 Zielgruppen

Die Study-Planner-Anwendung wird hauptsächlich von Studierenden genutzt. Sie planen damit ihre Veranstaltungen, Prüfungen und Lernzeiten und sind die einzige untersuchte Nutzerrolle. Weitere Rollen, zum Beispiel Administrator*innen oder Dozierende mit eigenen Zugriffsrechten, sind in dieser Version nicht vorgesehen.

2.3 Produktumgebung

Die Study-Planner-Anwendung wird als Desktop-Programm auf Einzelplatzrechnern betrieben. Sie benötigt eine installierte Java-Laufzeitumgebung (mindestens Java SE 17 oder eine im Projekt vereinbarte Version) unter:

- **Windows 10 oder höher.**
- einer aktuellen **Linux-Distribution**.

Weitere Hardwareanforderungen bestehen nicht; ein Standard-Laptop mit üblicher Bildschirmauflösung ist ausreichend. Die Anwendung speichert alle Daten lokal auf dem Rechner der Nutzer*innen.

2.3.1 Architektur

Die Anwendung folgt einer schichtenorientierten Architektur:

- **Präsentationsschicht**: grafische Benutzeroberfläche für die Verwaltung von Modulen, Terminen und Lernblöcken.
- **Anwendungsschicht**: Geschäftslogik zur Prüfung von Eingaben, Berechnung von Ansichten und Verwaltung der Planungsdaten.
- **Persistenzschicht**: Laden und Speichern der Daten in einer lokalen Datei bzw. einem lokalen Datenformat.

Es wird keine Client-Server-Struktur eingesetzt; der Study-Planner läuft als eigenständige Einzelplatzanwendung ohne Netzwerkkommunikation.

2.3.2 Technologien

Die Implementierung erfolgt in **Java SE**. Für die grafische Benutzeroberfläche wird eine in Java verfügbare GUI-Bibliothek verwendet, die im Projektteam festgelegt wird. Zum Bauen und Ausführen des Projekts wird eine gängige Java-Entwicklungsumgebung genutzt; dies hat jedoch keinen Einfluss auf den Produktiveinsatz beim Nutzer.

2.3.3 Komponenten

Logisch gliedert sich das System in folgende Hauptkomponenten:

- **UI-Komponente**: Dialoge und Masken zur Eingabe und Anzeige von Modulen, Terminen, Lernblöcken und Kalenderansichten.
- **Planungskomponente**: Verwaltung der Objekte (Module, Termine, Lernblöcke), Filterung nach Zeitraum/Modul und Bereitstellung der Daten für die Ansichten.
- **Erinnerungskomponente**: Verwaltung und Auslösung von Erinnerungen auf Basis der gespeicherten Termine.
- **Speicherkomponente**: Lesen und Schreiben der Planungsdaten in das lokale Speicherformat.

Weitere Unterkomponenten können im Design detailliert werden, ändern aber die grundsätzliche Struktur nicht.

2.3.4 Schnittstellen

Das System besitzt in der ersten Version keine externen Schnittstellen zu Fremdsystemen.

- Es nutzt ausschließlich das **lokale Dateisystem** zum Speichern und Laden der Planungsdaten.
- Für Erinnerungsfunktionen wird die **Systemzeit** des Betriebssystems verwendet.
- Eine Anbindung an Online-Kalender oder Web-APIs ist nicht vorgesehen und gehört nicht zum Lieferumfang der aktuellen Version.

2.4 Betriebsbedingungen

Die Study-Planner-Anwendung wird auf den privaten oder hochschuleigenen Rechnern der Studierenden betrieben. Die Nutzung erfolgt interaktiv durch die angemeldeten Nutzer*innen; ein unbeaufsichtigter Dauerbetrieb ist nicht vorgesehen. Eingesetzt wird das System in typischen Lern- und Arbeitsumgebungen. Es bestehen keine besonderen Anforderungen an Raumklima oder Stromversorgung über die üblichen Bedingungen eines Arbeitsplatzrechners hinaus.

3 Produktfunktionen/Anforderungen

3.1 Funktionale Anforderungen

3.1.1 Beschreibung der FA mit Rollen innerhalb der Geschäftsprozesse

Die folgenden Anwendungsfälle beschreiben die wichtigsten Funktionen der Study-Planner-Anwendung aus Sicht der Studierenden. Hauptnutzer ist jeweils eine studierende Person, die ihren Studienalltag planen und verwalten möchte.

AF-01	Module verwalten	Studierende*r legt für ein neues Semester Module an. Für jedes Modul werden Name, Semester und optional eine kurze Beschreibung eingegeben. Vorhandene Module können später bearbeitet oder gelöscht werden.
AF-02	Termine erfassen	Studierende*r erfasst studienbezogene Termine wie Vorlesungen, Übungen, Prüfungen oder Abgabefristen. Für jeden Termin werden mindestens Modul, Datum, Uhrzeit und eine kurze Bezeichnung gespeichert. Termine können geändert, gelöscht sowie als „erledigt“ oder wieder als „offen“ markiert werden. Der aktuelle Status wird in der Oberfläche sichtbar dargestellt.
AF-03	Lernblöcke planen	Zur Vorbereitung auf Prüfungen plant Studierende*r Lernblöcke. Dabei werden ein Modul, Datum, Startzeit und Dauer festgelegt und der Lernblock gespeichert. Geplante Lernblöcke können anschließend angepasst, entfernt sowie als „erledigt“ oder wieder als „offen“ markiert werden. Der aktuelle Status wird in der Oberfläche sichtbar dargestellt.
AF-04	Kalender anzeigen & filtern	Studierende*r lässt sich Termine und Lernblöcke in einer Tages- oder Wochenansicht anzeigen. Die Ansicht kann nach Zeitraum und Modul gefiltert werden.
AF-05	Erinnerungen erhalten	Für ausgewählte Termine aktiviert Studierende*r Erinnerungen. Vor Erreichen des Termins zeigt die Anwendung ein Pop-up mit den wichtigsten Informationen, z. B. „Prüfung Algorithmen morgen um 10:00 Uhr“, an. Der Vorlauf der Erinnerung kann eingestellt werden.

AF-06	Daten speichern & laden	Beim Beenden der Anwendung werden alle Planungsdaten (Module, Termine, Lernblöcke, Einstellungen) automatisch lokal gespeichert. Beim nächsten Start lädt die Anwendung diese Daten und stellt den zuletzt bekannten Zustand wieder her.
AF-07	Daten importieren & exportieren (optional)	Sofern im Projektumfang vorgesehen, können Planungsdaten in eine lokale Datei exportiert und aus einer solchen Datei wieder importiert werden.

3.1.2 Aktivitäten mit Benutzerschnittstelle (UI)

Anwendungsfall ID	AF-01
AF Name	Module verwalten
Akteur	Studierende*r
Vorbedingung	- Die Anwendung ist gestartet.
Auslösendes Ereignis	Nutzer*in möchte ein neues Modul hinzufügen oder ein bestehendes Modul bearbeiten.
Nachbedingung Erfolg	- Das Modul ist gespeichert und wird in der Modulübersicht angezeigt. Änderungen oder Löschungen werden übernommen.
Nachbedingung Fehlschlag	- Das Modul wird nicht gespeichert und eine Fehlermeldung wird angezeigt.
Ablauf (Erfolgsszenario „Neu“)	1. Nutzer*in öffnet die Modulübersicht. 2. Wählt **„Neues Modul“** oder ein bestehendes Modul zur Bearbeitung aus. 3. System öffnet den Dialog zur Moduleingabe. 4. Eingabe von Modulname, Semester und optionaler Informationen. 5. Klick auf **„Speichern“**. 6. System prüft Eingaben. 7. System speichert das Modul und aktualisiert die Modulübersicht.
Benutzerschnittstelle	- Modulübersicht mit bestehenden Modulen sowie Eingabedialog zum Erstellen und Bearbeiten von Modulen.

Anwendungsfall ID	AF-02
AF Name	Termine erfassen
Akteur	Studierende*r
Vorbedingung	- Die Anwendung ist gestartet. - Mindestens ein Modul ist angelegt.
Auslösendes Ereignis	Nutzer*in öffnet die Termin oder Kalenderansicht und klickt auf **„Neuer Termin“** oder Doppelklick auf ein Datum/Zeitfeld.
Nachbedingung Erfolg	- Der Termin ist gespeichert und wird in Terminliste sowie Kalenderansicht angezeigt. - Der aktuelle Status (**„offen“** oder **„erledigt“**) wird gespeichert und sichtbar dargestellt.
Nachbedingung Fehlschlag	- Kein neuer Termin wird gespeichert. - Bei fehlenden/ungültigen Angaben zeigt das System eine Fehlermeldung und erlaubt Korrektur.
Ablauf (Erfolgsszenario „Neu“)	1. Nutzer*in öffnet Termin oder Kalenderansicht. 2. Klick auf **„Neuer Termin“**. 3. Dialog „Termin bearbeiten“ öffnet sich. 4. Auswahl eines Moduls; Eingabe von Datum, Startzeit, Dauer/Endzeit. 5. Auswahl einer Terminart. 6. Klick auf **„Speichern“**. 7. System prüft Eingaben. 8. System speichert Termin und zeigt ihn in Liste/Kalender an. 9. Nutzer*in kann einen bestehenden Termin als **„erledigt“** markieren. 10. Das System aktualisiert den Status des Termins und speichert die Änderung. 11. Ein bereits erledigter Termin kann wieder als **„offen“** markiert werden. 12. Das System aktualisiert die Anzeige entsprechend des aktuellen Status.
Benutzerschnittstelle	- Kalenderansicht, z. B. Wochenansicht, mit Termin-Kacheln und Button **„Neuer Termin“**. - Alternativ Terminliste mit Spalten Datum, Zeit, Modul, Art, Titel. - Dialog „Termin bearbeiten“ mit Feldern für Modul, Datum, Zeit, Dauer, Terminart, Notiz sowie Buttons **„Speichern“**, **„Abbrechen“**. - Für bestehende Termine steht eine Aktion zur Statusänderung (**„offen“** / **„erledigt“**) zur Verfügung. Erledigte Termine werden sichtbar gekennzeichnet.

Anwendungsfall ID	AF-03
AF Name	Lernblöcke planen
Akteur	Studierende*r
Vorbedingung	- Die Anwendung ist gestartet. - Mindestens ein Modul ist angelegt.
Auslösendes Ereignis	Nutzer*in wählt im Hauptfenster den Bereich „Lernblöcke“ oder eine entsprechende Ansicht und klickt auf **„Neuer Lernblock“**.
Nachbedingung Erfolg	- Der Lernblock ist gespeichert und wird in der Liste bzw. Kalenderansicht angezeigt. - Der aktuelle Status (**„offen“** oder **„erledigt“**) wird gespeichert und sichtbar dargestellt.
Nachbedingung Fehlschlag	- Kein neuer Lernblock wird gespeichert. - Bei ungültigen Angaben zeigt das System eine Fehlermeldung und erlaubt Korrektur.
Ablauf (Erfolgsszenario „Neu“)	1. Nutzer*in öffnet den Bereich „Lernblöcke“ 2. Klick auf **„Neuer Lernblock“**. 3. Dialog „Lernblock bearbeiten“ öffnet sich. 4. Auswahl eines Moduls; Eingabe von Datum, Startzeit, Dauer. 5. Optional Eingabe einer Beschreibung, z. B. Thema des Lernblocks 6. Klick auf **„Speichern“**. 7. System prüft Eingaben. 8. System speichert den Lernblock und zeigt ihn in Liste/ Kalender an. 9. Nutzer*in kann einen bestehenden Lernblock als **„erledigt“** markieren. 10. Das System aktualisiert den Status des Lernblocks und speichert die Änderung. 11. Ein bereits erledigter Lernblock kann wieder als **„offen“** markiert werden. 12. Das System aktualisiert die Anzeige entsprechend des aktuellen Status.
Benutzerschnittstelle	- Bereich „Lernblöcke“ mit Liste oder Kalenderansicht geplanter Lernzeiten und Button **„Neuer Lernblock“**. - **„Bearbeiten“**/**„Löschen“** pro Eintrag - Dialog „Lernblock bearbeiten“ mit Auswahlfeld für Modul, Feldern für Datum, Startzeit, Dauer und einem Textfeld für die Beschreibung, Buttons **„Speichern“**, **„Abbrechen“**. - Für bestehende Lernblöcke steht eine Aktion zur Statusänderung (**„offen“** / **„erledigt“**) zur Verfügung. Erledigte Lernblöcke werden sichtbar gekennzeichnet.

Anwendungsfall ID	AF-04
AF Name	Kalender anzeigen und filtern
Akteur	Studierende*r
Vorbedingung	- Die Anwendung ist gestartet. - Es existiert mindestens ein Modul oder Termin bzw. Lernblock.
Auslösendes Ereignis	Nutzer*in wechselt in die **Kalenderansicht** oder ändert Filtereinstellungen (Zeitraum, Modul).
Nachbedingung Erfolg	- Kalenderansicht wird mit den gefilterten Terminen und Lernblöcken aktualisiert.
Nachbedingung Fehlschlag	- Bei ungültigen Filterangaben wird ein Standardzeitraum, z. B. aktuelle Woche, angezeigt oder der Filter zurückgesetzt.
Ablauf (Erfolgsszenario „Neu“)	1. Nutzer*in öffnet die Kalenderansicht. 2. Auswahl des gewünschten Zeitraums. 3. Optional Auswahl eines Moduls zur Filterung. 4. System lädt alle passenden Termine und Lernblöcke. 5. System zeigt Einträge in der Kalenderansicht an. 6. Nutzer*in kann zwischen Tages-, Wochen- oder Listenansicht wechseln.
Benutzerschnittstelle	- Kalenderbereich mit Navigations-Elementen, z. B. „Heute“, „Vorherige Woche“, „Nächste Woche“. - Dropdown oder Filterleiste für Modul-Auswahl. - Umschaltknöpfe für Tages-/Wochen-/Listenansicht. - Kalenderfläche, in der Termine und Lernblöcke als Blöcke dargestellt werden.

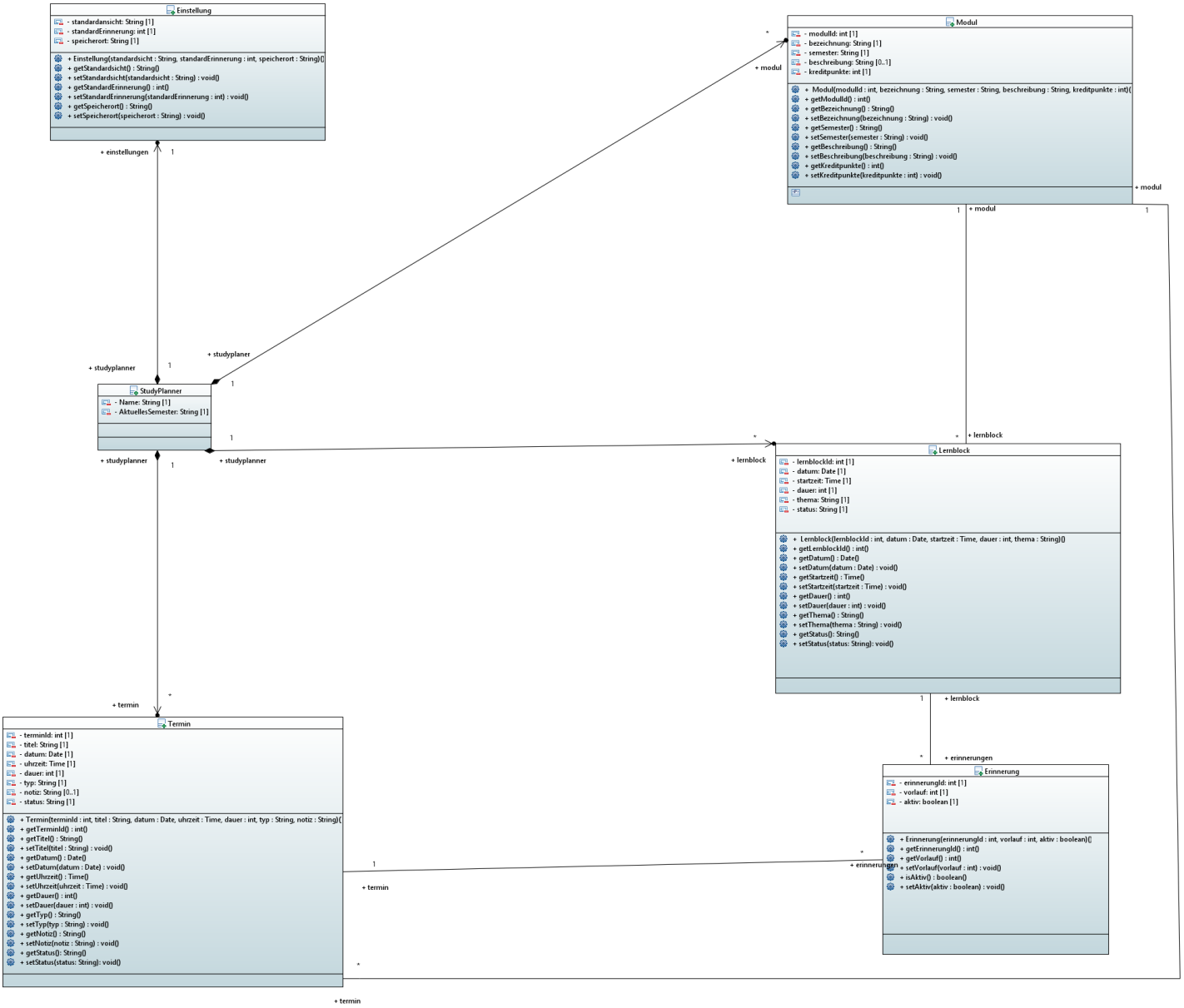
Anwendungsfall ID	AF-05
AF Name	Erinnerungen erhalten
Akteur	Studierende*r
Vorbedingung	- Die Anwendung ist gestartet. - Es existieren bereits Termine oder Lernblöcke.
Auslösendes Ereignis	Nutzer*in öffnet einen bestehenden Termin/Lernblock oder legt einen neuen an und aktiviert dort die Erinnerungsfunktion.
Nachbedingung Erfolg	- Für den gewählten Termin/Lernblock ist eine Erinnerung mit einem definierten Vorlauf, z. B. 10 Min., 1 Stunde, 1 Tag, gespeichert. - Zum definierten Zeitpunkt zeigt das System eine Benachrichtigung an.
Nachbedingung Fehlschlag	- Es wird keine Erinnerung gespeichert oder eine bestehende Erinnerung wird deaktiviert. - Bei ungültigen Einstellungen wird eine Fehlermeldung angezeigt und die alte Konfiguration beibehalten.
Ablauf (Erfolgsszenario „Neu“)	1. Nutzer*in öffnet Dialog „Termin bearbeiten“ oder „Lernblock bearbeiten“ 2. Aktiviert Checkbox „Erinnerung aktivieren“. 3. Wählt einen Vorlauf aus . 4. Klick auf **„Speichern“**. 5. System speichert Termin/Lernblock inklusive Erinnerungseinstellungen. 6. Zum berechneten Zeitpunkt löst das System eine Benachrichtigung aus.
Benutzerschnittstelle	- In den Dialogen „Termin bearbeiten“ / „Lernblock bearbeiten“ ein Bereich „Erinnerung“ mit Checkbox **„Erinnerung aktivieren“** und Dropdown für den Vorlauf. - Pop-up / Hinweisfenster, das zum Erinnerungszeitpunkt den Termin/Lernblock mit Datum, Uhrzeit und Modul anzeigt; Buttons z. B. **„Schließen“**, optional **„Später erinnern“**.

Anwendungsfall ID	AF-06
AF Name	Daten speichern & laden
Akteur	Studierende*r
Vorbedingung	Die Anwendung ist gestartet; die Standard-Speicherposition ist vorhanden oder im System konfiguriert.
Auslösendes Ereignis	- Automatisch beim Beenden der Anwendung oder in festgelegten Intervallen **oder** - manuell durch Auswahl von **„Datei → Speichern“** bzw. **„Datei → Öffnen“**.
Nachbedingung Erfolg	- Alle aktuellen Planungsdaten (Module, Termine, Lernblöcke, Einstellungen) sind in der lokalen Datei gespeichert. - Beim Laden werden die Daten vollständig eingelesen und im System angezeigt.
Nachbedingung Fehlschlag	- Beim Speichern: Daten werden nicht oder nur teilweise geschrieben; System zeigt Fehlermeldung an und verwirft den Vorgang nicht kommentarlos. - Beim Laden: Datei kann nicht gelesen werden; System zeigt Fehlermeldung und behält den bisherigen Zustand bei.
Ablauf (Erfolgsszenario „Neu“)	1. Nutzer*in beendet die Anwendung, z. B. über „Schließen“-Button oder Menü „Datei → Beenden“. 2. System sammelt alle aktuellen Planungsdaten. 3. System schreibt die Daten in die konfigurierte lokale Datei. 4. System meldet keinen Fehler und beendet sich.
Benutzerschnittstelle	- Menüleiste mit Einträgen **„Datei → Neu“**, **„Öffnen…“**, **„Speichern“**, **„Beenden“**. - Standard-Dateiauswahldialog des Betriebssystems. - Optional Statusmeldung im unteren Bereich (z. B. „Daten erfolgreich gespeichert“ oder Fehlermeldung bei Problemen).

Anwendungsfall ID	AF-07
AF Name	Daten importieren & exportieren (optional)
Akteur	Studierende*r
Vorbedingung	- Die Anwendung ist gestartet. - Für den Export: Es liegen Planungsdaten (Module, Termine, Lernblöcke) vor. - Für den Import: Eine gültige Export- oder Austauschdatei liegt lokal vor.
Auslösendes Ereignis	Nutzer*in wählt im Menü **„Datei → Export…“** oder **„Datei → Import…“**.
Nachbedingung Erfolg	- Export: Eine Datei mit allen relevanten Planungsdaten wurde erfolgreich erstellt. - Import: Die Daten aus der gewählten Datei wurden gelesen und in das System übernommen (Module, Termine, Lernblöcke).
Nachbedingung Fehlschlag	- Export: Datei konnte nicht geschrieben werden; System zeigt eine Fehlermeldung. - Import: Datei ist ungültig oder kann nicht gelesen werden; System zeigt eine Fehlermeldung und verändert den aktuellen Datenbestand nicht.
Ablauf (Erfolgsszenario „Neu“)	1. Nutzer*in wählt **„Datei → Export…“**. 2. System öffnet einen Dateiauswahldialog und schlägt einen Dateinamen vor (z. B. studyplanner_export.spd). 3. Nutzer*in wählt Speicherort und bestätigt. 4. System sammelt aktuelle Planungsdaten (Module, Termine, Lernblöcke, Einstellungen). 5. System schreibt die Daten in die Exportdatei. 6. System zeigt eine Erfolgsmeldung, z. B. „Daten erfolgreich exportiert“.
Benutzerschnittstelle	- Menüeinträge **„Datei → Export…“** und **„Datei → Import…“**. - Standard-Dateiauswahldialog zum Auswählen von Speicherort bzw. Importdatei. - Kurze Status-/Fehlermeldungen im Hauptfenster (z. B. unten in einer Statusleiste): „Export erfolgreich“, „Import fehlgeschlagen – ungültiges Format“ usw.

3.1.3 Fachliches Klassendiagramm (Domain Model) / Produktdaten

Das fachliche Klassendiagramm beschreibt die zentralen Datenobjekte der Study-Planner-Anwendung sowie deren Beziehungen zueinander. Es dient als Grundlage für die Strukturierung der Anwendungslogik und zeigt die wichtigsten Attribute und Operationen der einzelnen Klassen.



Die Anwendung basiert auf den zentralen Klassen StudyPlanner, Modul, Termin, Lernblock, Erinnerung und Einstellung. Module dienen der Organisation des Studiums und können mit Terminen sowie Lernblöcken verknüpft werden. Termine und Lernblöcke enthalten neben zeitlichen Angaben einen Status, über den der aktuelle Bearbeitungszustand („offen“ oder „erledigt“) abgebildet wird. Erinnerungen ermöglichen eine zeitbezogene Benachrichtigung zu geplanten Aktivitäten, während Einstellungen individuelle Konfigurationen der Anwendung speichern.

3.2 Nichtfunktionale Anforderungen

3.2.1 Benutzbarkeit

- Die Study-Planner-Anwendung soll eine **übersichtliche, reduzierte Oberfläche** bieten. Hauptfunktionen (Module, Termine, Lernblöcke, Kalender) sind über eine gut sichtbare Navigationsleiste erreichbar.
- Beschriftungen, Meldungen und Menüeinträge sind in **klarer, verständlicher Sprache** (Deutsch) zu formulieren, ohne unnötigen Fachjargon.
- Wichtige Aktionen wie *Speichern*, *Löschen* oder *Abbrechen* müssen stets eindeutig erkennbar und konsistent platziert sein.
- Fehlermeldungen sollen **konkret und hilfreich** sein, z. B. „Datum ist ungültig“ statt nur „Fehler“.
- Die Anwendung soll auf übliche Bildschirmauflösungen von Studierenden-Laptops ohne horizontales Scrollen nutzbar sein.

3.2.2 Zuverlässigkeit

- Die Anwendung darf bei normaler Nutzung **nicht abstürzen** oder Daten verlieren.
- Beim Beenden der Anwendung sind alle geänderten Daten entweder automatisch zu speichern oder es ist ein deutlicher Hinweis mit Speichermöglichkeit anzuzeigen.
- Tritt beim Lesen oder Schreiben von Dateien ein Fehler auf, muss das System eine **Fehlermeldung mit Ursache** anzeigen und den bisherigen Datenstand nicht überschreiben.
- Interne Fehler (z. B. unerwartete Ausnahmen) sind zu protokollieren, sodass eine spätere Analyse möglich ist.

3.2.3 Effizienz

- Die Anwendung soll auf typischen Studierenden-Rechnern **innerhalb von höchstens 5 Sekunden** startbereit sein.
- Standardaktionen wie das Öffnen des Kalenders, das Anlegen eines Lernblocks oder das Umschalten von Filtern sollen bei einer typischen Datenmenge von bis zu **500 Einträgen** in **unter 1 Sekunde** reagieren.
- Hintergrundoperationen dürfen die Interaktion mit der Oberfläche nicht merklich blockieren.

3.2.4 Softwarewartung

- Der Quellcode ist in **klare Schichten** (Präsentation, Fachlogik, Persistenz) zu gliedern, sodass Änderungen an einer Schicht möglichst keine Anpassungen in den anderen erzwingen.
- Klassen, öffentliche Methoden und zentrale Datenstrukturen sind mit kurzen, aussagekräftigen Kommentaren zu dokumentieren.
- Konfigurierbare Einstellungen (z. B. Speicherort der Daten, Standard-Erinnerungsvorlauf) sollen, soweit sinnvoll, in einer **externen Konfigurationsdatei** gehalten werden.
- Das System soll so aufgebaut sein, dass spätere Erweiterungen (z. B. zusätzliche Erinnerungsarten oder weitere Ansichten) ohne grundlegende Umstrukturierung möglich sind.

3.2.5 Sicherheit

- Alle Planungsdaten (Module, Termine, Lernblöcke, Einstellungen) werden **lokal auf dem Gerät der Studierenden** gespeichert; die Anwendung baut von sich aus keine Netzwerkverbindungen auf.
- Beim Zugriff auf Dateien sind nur Verzeichnisse zugelassen, die von Nutzer*innen explizit ausgewählt oder in den Einstellungen konfiguriert wurden.
- Ungültige oder manipulierte Datendateien dürfen nicht zu Programmabstürzen führen; das System muss eine verständliche Fehlermeldung anzeigen und den aktuellen Datenbestand unverändert lassen.
- Es sollen keine sensiblen Daten wie Passwörter oder Zugangsdaten zu anderen Diensten gespeichert werden.

3.2.6 Normen

NF-B1	Benutzung	Die Anwendung ist für die Nutzung mit Maus und Tastatur auf einem Desktop- bzw. Laptop-Rechner vorgesehen.
NF-E1	Effizienz	Standardaktionen sollen bei typischer Datenmenge in unter 1 Sekunde reagieren.
NF-W1	Softwarewartung	Der Quellcode ist in Schichten (UI, Fachlogik, Persistenz) zu strukturieren, um Änderungen und Erweiterungen zu erleichtern.
NF-S1	Sicherheit	Alle Planungsdaten werden lokal gespeichert; die Anwendung baut ohne ausdrückliche Konfiguration keine Netzwerkverbindungen auf.
NF-N1	Normen	Es sind keine speziellen branchenspezifischen Normen zu erfüllen; allgemeine Programmier- und Usability-Richtlinien der Hochschule sind zu beachten.

4 Testung

Zur Sicherstellung der Qualität der Study-Planner-Anwendung werden sowohl manuelle Funktionstests als auch automatisierte Modultests durchgeführt.

Die manuellen Funktionstests überprüfen die vollständigen Abläufe aus Sicht der Nutzer*innen. Dabei werden insbesondere die in Kapitel 3 beschriebenen Anwendungsfälle wie die Verwaltung von Modulen, Terminen und Lernblöcken getestet.

Zusätzlich werden automatisierte Modultests mit JUnit 5 eingesetzt. Diese prüfen einzelne Komponenten der Fachlogik unabhängig voneinander und stellen sicher, dass zentrale Funktionen wie das Erstellen, Bearbeiten und Löschen von Modulen, Terminen und Lernblöcken korrekt funktionieren.

Erweiterungen der Anwendung werden durch zusätzliche Testfälle abgesichert. Dazu gehört unter anderem die Prüfung, ob Termine und Lernblöcke korrekt zwischen den Status „offen“ und „erledigt“ wechseln und die Änderung gespeichert wird.

5 Monitoring / Support bei Übergabe oder ähnliche Leistungen

Nach der Übergabe der Study-Planner-Anwendung werden folgende Unterstützungsleistungen vereinbart:

- **Bereitstellung des Quellcodes und der Projektdokumentation** (Pflichtenheft, Installationshinweise, kurze Gebrauchsanleitung).
- **Fehlerbehebung im Semester der Abnahme** – gemeldete Fehler, die die Nutzung der Kernfunktionen (Modulverwaltung, Termine, Lernplanung) deutlich beeinträchtigen, werden analysiert und in einer aktualisierten Version behoben.
- **Begrenzter E-Mail-Support** – eine Kontaktadresse der Cognivia GmbH wird bereitgestellt; einfache Rückfragen zur Installation oder Bedienung werden im Rahmen des Projekts beantwortet.

Ein dauerhaftes technisches Monitoring, z. B. Serverüberwachung, ist nicht vorgesehen, da die Anwendung lokal auf den Geräten der Studierenden betrieben wird.

6 Dokumentation

Im Rahmen des Projektes werden verschiedene Dokumente erstellt, die die Benutzung, Wartung und Weiterentwicklung der Study-Planner-Anwendung unterstützen.

6.1 Anwenderdokumentation

Die Anwenderdokumentation beschreibt aus Sicht der Studierenden die Bedienung der Anwendung. Sie enthält insbesondere:

- kurze **Installationsanleitung** (Starten der Anwendung, notwendige Umgebung),
- Überblick über die **Hauptbereiche** (Modulliste, Kalender, Lernblöcke),
- Schritt-für-Schritt-Anleitungen zum
    - Anlegen, Bearbeiten und Löschen von Modulen,
    - Erfassen von Terminen und Lernblöcken,
    - Markieren von Terminen und Lernblöcken als „erledigt“ bzw. „offen“,
    - Setzen und Verwalten von Erinnerungen,
- Hinweise zum Speichern und Sichern der Daten,
- Zusammenfassung häufig auftretender Fehler und deren Behebung.

6.2 Administratorendokumentation

Die Administratorendokumentation beschreibt die Installation und technische Einrichtung der Anwendung. Sie enthält Informationen zu benötigten Java-Versionen, Systemvoraussetzungen, Start der Anwendung sowie Hinweise zur Ablage und Sicherung der lokalen Daten.

6.3 Entwicklerdokumentation

Die Entwicklerdokumentation dient zur Unterstützung der Wartung und Weiterentwicklung der Anwendung. Sie umfasst die Beschreibung der Systemarchitektur, die erstellten UML-Diagramme sowie Informationen zur Projektstruktur.
Zusätzlich werden Build- und Einrichtungshinweise dokumentiert, um die Entwicklungsumgebung nachvollziehbar einzurichten. Die technische Dokumentation des Quellcodes erfolgt mithilfe von Javadoc. Ergänzende Informationen wie Projektaufbau, verwendete Technologien und Startanweisungen werden über Markdown-Dateien (z. B. README) im Repository bereitgestellt.

6.4 Weitere referenzierte Dokumente

Das Pflichtenheft wurde in enger Anlehnung an das zuvor erstellte Lastenheft der InnoWeb Solutions GmbH für den Study Planner ausgearbeitet. Folgende Dokumente sind ergänzend relevant und im Projekt-Repository hinterlegt:

- Lastenheft des Auftraggebers,
- Firmenporträts von Auftraggeber und Auftragnehmer,
- Angebot und Projektplan der Cognivia GmbH,
- UML-Modelle (Klassendiagramm, ggf. weitere Diagramme),
- Testfälle und Testprotokolle.

Diese Unterlagen bilden zusammen mit dem Pflichtenheft die Grundlage für Implementierung, Abnahme und mögliche spätere Erweiterungen des Systems.

7 Vorgehen

Für die Umsetzung der Study-Planner-Anwendung wird ein **inkrementell-iteratives Vorgehen** gewählt. Zunächst wird ein lauffähiger Basis-Prototyp mit den wichtigsten Kernfunktionen erstellt (Modulverwaltung und Terminübersicht). In weiteren Iterationen werden zusätzliche Funktionen (Lernblöcke, Erinnerungen, Filter/Ansichten) sowie nichtfunktionale Anforderungen (Bedienbarkeit, Performance, Wartbarkeit) ergänzt und verfeinert.

Jede Iteration liefert eine testbare Version. Die letzte Iteration vor der Abnahme gilt als *Release Candidate*; auf dieser Basis werden nur noch Fehlerkorrekturen vorgenommen (Code Freeze) und die Projektdokumentation abgeschlossen. Anschließend erfolgt die Übergabe an den Auftraggeber.

Datum	Meilenstein
16.03.2026	Projektauftakt, Abstimmung von Umfang und Rollen
30.03.2026	Fertigstellung grober Architekturentwürfe
20.04.2026	Prototyp Basisfunktionen (Module anlegen, Termine planen)
04.05.2026	Erweiterung um Lernblöcke und einfache Erinnerungen
01.06.2026	Erweiterung der Anwendung, Aktualisierung von Tests und Dokumentation
29.06.2026	Finale Abnahme und Übergabe der Anwendung

Die Fortschrittskontrolle erfolgt in der Regel wöchentlich:

- Zu jedem Termin wird der aktuelle Stand kurz vorgestellt (fertige Funktionen, offene Punkte, nächste Schritte).
- Wichtige Zwischenergebnisse wie Prototypen, implementierte Anwendungsfälle oder aktualisierte Dokumente (z. B. Klassendiagramm, Testfälle) werden als eigenständige Arbeitspakete betrachtet.
- Für jede Iteration werden Soll-Ziele (geplante Funktionen/Artefakte) festgelegt; im Projektprotokoll wird festgehalten, welche Ziele erreicht wurden (Ist-Stand) und welche Aufgaben in die nächste Iteration verschoben werden.

Die Darstellung des Projektfortschritts kann tabellarisch in einem kurzen Projektbericht erfolgen, z. B. Liste von Arbeitspaketen mit Status: „geplant / in Arbeit / fertig“. Wichtig ist, dass die Dokumentation regelmäßig aktualisiert wird und für den Auftraggeber jederzeit erkennbar ist, wie weit die Implementierung der Study-Planner-Anwendung fortgeschritten ist.

8 Entwicklungsumgebung

Für die Entwicklung der Study-Planner-Anwendung wird eine einheitliche Entwicklungsumgebung eingesetzt. Die Auswahl der Werkzeuge basiert auf einem vorherigen Toolvergleich und berücksichtigt insbesondere Integration, Teamfähigkeit und Eignung für Java-Projekte.

Folgende Werkzeuge werden verwendet:

Versionierung	GitLab
UML-Modellierung	Papyrus
Build-Management	Maven
GUI-Prototyping	Figma
Entwicklungsumgebung	Eclipse, Visual Studio Code
Testautomatisierung	JUnit 5
Dokumentation	Javadoc, Markdown
Obfuscation	ProGuard
Codekonvention	Google Java Style Guide
Zusammenarbeit	GitLab, WhatsApp, Präsenzabstimmungen

9 Glossar

AF:	Anwendungsfall (Use Case) / in sich geschlossene Interaktion zwischen einer Benutzerrolle und dem System mit einem klar definierten fachlichen Ziel.
FA:	Funktionale Anforderungen / Anforderungen, die das beobachtbare Verhalten der Software beschreiben, z. B. Anlegen von Modulen, Planen von Lernblöcken, Export von Terminen.
NF-xx:	Nichtfunktionale Anforderungen / Qualitätsanforderungen an das System, z. B. Benutzbarkeit, Wartbarkeit und Sicherheit.
MK-xx:	Muss-Kriterium / zwingende Anforderung an das System, deren Erfüllung für die erfolgreiche Abnahme erforderlich ist.
KK-xx:	Kann-Kriterium / optionale Anforderung, deren Umsetzung wünschenswert ist, aber nicht zwingend für die Abnahme erforderlich ist.
AK-xx:	Abgrenzungskriterium / bewusst nicht verfolgte Anforderungen bzw. Funktionsbereiche, um den Projektumfang klar einzugrenzen.
UI:	User Interface (Benutzerschnittstelle) / grafische Oberfläche der Study-Planner-Anwendung zur Eingabe, Anzeige und Verwaltung von Planungsdaten.
Prototyp:	Frühe, funktionsreduzierte Version des Systems, mit der Kernfunktionen und Bedienkonzepte validiert werden.
Release Candidate:	Nahezu fertige Systemversion, die alle Muss-Kriterien erfüllt und für abschließende Tests sowie die Abnahme durch den Auftraggeber vorgesehen ist.
Repository:	Zentraler, versionsverwalteter Speicherort für Quellcode und Projektdokumente, der der gemeinsamen Entwicklung und Nachvollziehbarkeit von Änderungen dient.